datagraph & impactgraph — Technical Reference

Implementation documentation, module by module, function by function.

Engine	`datagraph` — PyPI distribution `datagraph-core` **0.8.4**, import name `datagraph`, CLI `datagraph`
PR product	`impactgraph` — PyPI `impactgraph` **0.7.3**, import name `impactgraph`, CLI `impactgraph`
Repositories	github.com/sumit-gupta03/datagraph · github.com/sumit-gupta03/impactgraph
Licence	MIT
Python	3.9 – 3.13 (CI matrix), developed on 3.14
Size	datagraph ≈ 5,100 lines of package code + 2,100 lines of tests (145 tests); impactgraph ≈ 320 lines + 126 lines of tests (6 tests)
Hard dependency	`networkx` only. Everything else is an optional extra.

How to read this document. Part I explains the architecture and the two files that everything else depends on (`graph/model.py`, `graph/graph.py`). Part II documents every extractor — the code that turns artifacts into graph fragments. Part III documents every analysis, output and service module. Part IV documents impactgraph, the test suite, CI/CD and packaging. Appendices hold the complete CLI, API, node-id and environment-variable references.

Every section names the file, the functions with their line numbers (as of 0.8.4 / 0.7.3), what the code does, *why* it does it that way, and which test proves it.

Table of contents

Part I — Architecture and core 1. Design principles 2. Repository layout 3. Data flow end to end 4. `graph/model.py` — nodes, edges, direction, provenance 5. Node id conventions 6. `graph/graph.py` — the graph engine

Part II — Extractors (7–20) · **Part III — Analysis, outputs, services** (21–33) · **Part IV — impactgraph, tests, CI/CD** (34–41) · **Appendices** (A–F)

Part I — Architecture and core

1. Design principles

Five rules drove every implementation decision. They are worth stating first because most of the code is a consequence of them.

1.1 The graph is never built by an LLM. Nodes and edges come from artifacts that already exist: Python's own `ast` module, dbt's `manifest.json`, SQL parsed by `sqlglot`, `information_schema`, `git diff`, OpenLineage events, DataHub's GraphQL API. An LLM is optional and can only (a) *explain* a computed result or (b) *suggest* relationships that are then validated against existing nodes and tagged `llm`. Consequence: results are reproducible, reviewable and free, and the library is useful with no API key at all.

1.2 Every edge carries provenance. `extracted` (read from an artifact), `inferred` (a documented heuristic — name-resolved function call, same-name column, name-inferred foreign key), or `llm` (accepted suggestion). `--no-inferred` / `include_inferred=False` keeps only `extracted`. Consequence: a user can always ask "show me only what you actually know".

1.3 Direction is a property of the edge type, not of the traversal. A single table, `IMPACT_DIRECTION` (§4.3), says whether change flows along an edge or against it. Consequence: `impact()` is one BFS with no special cases, and adding a new edge type is a one-line change.

1.4 State is one JSON file. No server, no database, no daemon. `datagraph.json` is portable, diffable, cacheable in CI, and readable by both libraries. Consequence: `pip install` is the entire installation procedure.

1.5 Optional dependencies must degrade, never crash. `sqlglot`, `PyYAML`, `anthropic`, `boto3`, `mcp` are extras. Code paths that need them import *inside the function* and raise a message that names the extra to install. Consequence: the core install stays small (networkx + rich).

2. Repository layout

datagraph (`src/datagraph/`)

File	Lines	Responsibility
`__init__.py`	60	Public API re-exports; `__version__`
`graph/model.py`	107	`NodeType`, `EdgeType`, `IMPACT_DIRECTION`, `Node`, `Edge`, provenance constants
`graph/graph.py`	547	`ImpactGraph`: build, query, traverse, export, persist; `diff_graphs`
`extractors/base.py`	22	The `Extractor` protocol
`extractors/python_extractor.py`	232	Python AST → files, functions, classes, imports, calls, SQL-in-code
`extractors/sql_in_code.py`	89	Detects SQL inside string literals; returns (reads, writes, certain)
`extractors/sql_extractor.py`	257	sqlglot: table lineage, column lineage, `SELECT *` expansion
`extractors/dbt_extractor.py`	322	manifest.json + catalog.json → models, sources, exposures, columns, owners, tests, compiled-SQL column lineage
`extractors/warehouse_extractor.py`	255	`information_schema` / SQLite → tables, columns, PKs, FKs, view lineage; `connect()`
`extractors/git_extractor.py`	119	`git diff` → changed files and line ranges → node ids
`extractors/openlineage_extractor.py`	131	OpenLineage run events → datasets, jobs, column lineage, owners
`extractors/lineage_file_extractor.py`	139	DataHub-style YAML/JSON lineage files
`extractors/datahub_extractor.py`	163	Live DataHub GraphQL import
`extractors/airflow_extractor.py`	224	DAG files (AST) → DAGs, tasks, dependencies, callables, SQL
`extractors/lambda_extractor.py`	216	serverless.yml / SAM / CloudFormation → lambdas, handlers, APIs, events
`extractors/js_extractor.py`	147	JS/TS (regex) → files, functions, imports, calls, SQL-in-code
`extractors/registry.py`	95	Plugin registry + `datagraph.extractors` entry points
`analysis/impact.py`	86	`ImpactAnalysis` dataclass, `analyze_impact()`
`analysis/risk.py`	71	Weighted risk score → LOW/MEDIUM/HIGH/CRITICAL
`analysis/tests_recommender.py`	56	Rule-based test plan
`analysis/relationships.py`	65	Schema map: tables, columns, FK/lineage links
`analysis/modeling.py`	515	Kimball dimensional modelling
`profiling.py`	203	Row counts, freshness, nulls, distincts, min/max, top values; PII masking
`knowledge.py`	218	`context()` packs, `build_wiki()`, GRAPH_REPORT, llms.txt
`security.py`	97	DSN redaction, text sanitisation, prompt wrapping, PII detection, SQL/HTML escaping
`ai/providers.py`	198	`LLMProvider` + Anthropic / Bedrock / OpenAI-compatible backends
`ai/explain.py`	53	Plain-language explanation of an analysis

File	Lines	Responsibility
`ai/lineage.py`	163	Schema summary, lineage suggestions, `apply_suggestions()`
`report.py`	163	Rich terminal rendering with ASCII fallback
`html_report.py`	241	Self-contained interactive HTML (impact / lineage / whole graph)
`mcp_server.py`	121	9 MCP tools + stdio server
`maintenance.py`	110	Input fingerprints (`--update`), `watch` , git hooks
`cli.py`	832	22 commands, argument parsing, dispatch

impactgraph (`src/impactgraph/`)

File	Lines	Responsibility
`__init__.py`	19	Re-exports all of datagraph + `check` , `to_markdown` , `CheckResult`
`core.py`	183	`CheckResult` , `build_graph()` , `check()` , `to_markdown()` , `safe_console_text()`
`cli.py`	117	`check` / `pr` commands, passthrough to datagraph
`action.yml`	113	Composite GitHub Action

3. Data flow end to end

```
artifacts —▶ extractors —▶ ImpactGraph (merge) —▶ analyses —▶ outputs
```

1. **Extract.** Each extractor is independent and returns its own `ImpactGraph` *fragment*. No extractor knows about any other. They agree only on the node-id conventions (§5).
2. **Merge.** `graph.merge(fragment)` unions nodes and edges. Because ids are conventional, a dbt model and the Python function that writes its table meet at the same `table:` node without any coordination. `link_table_aliases()` (§6.12) then joins different qualifications of one relation.
3. **Analyse.** `impact()` , `upstream()` , `lineage()` , `hotspots()` walk the graph; `analyze_impact()` adds risk, owners and a test plan; `relationships()` , `star_schema()` , `profile_warehouse()` , `context()` read it from other angles.
4. **Output.** Terminal (rich), JSON, interactive HTML, Markdown wiki, GraphML/DOT/Cypher, MCP tools.

The graph is the only interface between the halves: adding an extractor makes every analysis and output work on the new data with no further changes, which is the main reason the design pays off.

4. `graph/model.py` — nodes, edges, direction, provenance

107 lines, no dependencies beyond `dataclasses` and `enum` . Everything else imports from here.

4.1 `NodeType` (L10) — 18 kinds of artifact

`file`, `module`, `function`, `class` (code) · `dbt_model`, `dbt_source`, `dbt_seed`, `dbt_snapshot`, `exposure` (dbt) · `table`, `view`, `column` (data) · `lambda`, `api`, `dag`, `task` (runtime) · `report`, `dashboard` (consumption).

It is a `str` enum, so `node.type.value` is a plain string in JSON and comparisons against strings work in templates.

4.2 `EdgeType` (L31) — 6 semantic relationships

Type	Direction convention	Emitted by
`contains`	parent → child	file→function, table→column, DAG→task
`calls`	caller → callee	Python/JS call resolution
`imports`	importer → imported	Python/JS imports
`depends_on`	downstream → upstream	dbt DAG, SQL lineage, FKs, aliases, LLM suggestions
`writes_to`	writer → target	dbt model → table, function → table, job → dataset
`exposes`	producer → exposure	model → dashboard

4.3 `IMPACT_DIRECTION` (L54) — the heart of the library

```
IMPACT_DIRECTION = {
    EdgeType.CONTAINS:   "forward",   # change the file  -> its functions are affected
    EdgeType.CALLS:      "reverse",   # change the callee -> the caller is affected
    EdgeType.IMPORTS:    "reverse",   # change the imported module -> the importer is affected
    EdgeType.DEPENDS_ON: "reverse",   # change the upstream model -> the downstream is affected
    EdgeType.WRITES_TO:  "forward",   # change the writer -> the written table is affected
    EdgeType.EXPOSES:    "forward",   # change the producer -> the dashboard is affected
}
```

Three of six edge types propagate *against* their own direction. That is the single fact that makes impact analysis correct: `a depends_on b` is stored downstream→upstream because that is how dbt expresses it, but a change to `b` is what affects `a`. Encoding this per type rather than per traversal is what keeps `impact()` free of special cases and makes `upstream()` its exact mirror.

4.4 Provenance constants (L64–66)

`EXTRACTED = "extracted"`, `INFERRED = "inferred"`, `LLM = "llm"`. Stored in `edge.meta["provenance"]`. Anything that is not `EXTRACTED` is dropped when `include_inferred=False`.

4.5 `Node` (L71) and `Edge` (L91)

```
@dataclass
class Node:
    id: str                 # conventional, unique  (see §5)
    type: NodeType
    name: str               # short display name
    path: Optional[str]     # repo-relative path when the node has one
    meta: Dict[str, Any]    # everything else: lineno, owner, schema, profile, sql, tests, ...
```

`Node.owner` (L79) is a property reading `meta["owner"]` so callers do not have to know where it lives. `to_dict()` / `from_dict()` (L82/L86) are the JSON contract — `meta` is stored verbatim, which is why extractors can attach anything (profiles, compiled SQL, test names) without a schema change.

```
@dataclass
class Edge:
    src: str; dst: str; type: EdgeType; meta: Dict[str, Any]
```

`Edge.provenance` (L98) is a property over `meta["provenance"]`, defaulting to `EXTRACTED`. `meta["via"]` carries a human explanation (`foreign_key` , `view_definition` , `sql-in-code` , `alias` , `llm`) which the terminal, HTML and Markdown renderers show verbatim.

5. Node id conventions

Ids are the coordination mechanism between extractors that never call each other, so the rules are strict and documented:

```
file:models/customer.sql                    a file, path relative to the scan root
func:src/api.py::customers_endpoint         a function; :: separates path from qualname
class:src/models.py::Customer               a class (qualname includes nesting: Outer.inner)
dbt:dim_customer                            a dbt model / seed / snapshot, by name
source:raw.customers                        a dbt source, "<source_name>.<name>"
exposure:revenue_report                     a dbt exposure / dashboard / report
table:prod.analytics.customer               a relation; catalog.schema.name, all lower-case
view:...                                    (views also use the table: prefix; NodeType marks them)
column:dim_customer.customer_key            "<parent bare id>.<column>", lower-case
job:airflow/load_dim_customer               an OpenLineage job: "<namespace>/<name>"
dag:nightly_bookings                        an Airflow DAG
task:nightly_bookings/build_dim             an Airflow task: "<dag>/<task_id>"
lambda:GetBookings                          an AWS Lambda function
api:GET /bookings                           an HTTP route
```

Two conventions matter more than they look:

- **Lower-casing table and column ids.** sqlglot upper-cases identifiers for some dialects; the warehouse returns whatever the engine stores. Without normalisation, `CUSTOMER_ID` from Snowflake and `customer_id` from dbt would be two nodes. Everything that creates a table/column id lower-cases it (`_table_id` , `_add_column` , dbt/SQL extractors). Tests: `test_column_impact.py` .

- **Column ids embed the parent's bare id**, so `column:dim_customer.customer_key` is derivable from `dbt:dim_customer`; `meta["parent"]` stores the full parent id for the reverse direction.

6. `graph/graph.py` — the graph engine

547 lines wrapping a `networkx.MultiDiGraph`. networkx stores the topology; the `Node` / `Edge` objects live in the node/edge attribute dictionaries (`self._g.nodes[id]["node"]`). This keeps our semantics (provenance, meta, types) independent of networkx while still getting its algorithms (`all_simple_paths`, `write_graphml`).

6.1 `add_node(node)` (L32) — merge-on-conflict, never overwrite

```
existing = self._g.nodes.get(node.id)
if existing is not None:
    existing["node"].meta.update({k: v for k, v in node.meta.items() if v is not None})
    if existing["node"].path is None and node.path:
        existing["node"].path = node.path
    return existing["node"]
```

Adding a node that already exists **enriches** it: non-`None` meta keys are merged, and a missing `path` is filled in. This is what allows the warehouse extractor to add types and the dbt extractor to add owners to the *same* table node, in either order. Overwriting would make the result depend on the order of `--warehouse` and `--dbt-manifest` flags.

6.2 `add_edge(edge)` (L42) — auto-create endpoints, dedupe, keep the strongest provenance

Three behaviours in fourteen lines:

1. **Endpoints are created on demand** via `_infer_node_from_id` (L522), which parses the id prefix into a `NodeType`. An extractor can therefore emit `Edge(func → table:analytics.orders)` without knowing whether the warehouse extractor has run.
2. **Deduplication by (src, dst, type)** — repeated runs or overlapping sources do not multiply edges.
3. **Provenance upgrade**: if the same edge exists as `inferred` and arrives again as `extracted`, the stored edge is *promoted*. A heuristic guess is silently replaced by hard evidence, never the other way round.

6.3 `merge(other)` (L56)

Nodes first, then edges, both through `add_node` / `add_edge` so the rules above apply. Fragments are therefore commutative and idempotent — `merge(a); merge(b)` equals `merge(b); merge(a)`. Test: `test_graph.py::test_merge_unifies_code_and_data`.

6.4 `find(query)` (L88) and `resolve(ref)` (L98)

`find` is a case-insensitive substring match over id, name and path — it returns *all* matches and is what `datagraph nodes --search` shows.

`resolve` turns a human reference into exactly one node, in a fixed priority order: exact id → unique exact name → unique exact path → the single FILE node on that path → unique substring match. If two nodes are equally good (the classic case: a dbt model `dim_customer` **and** the physical table it materialises), it returns `None` rather than guessing.

Callers then explain the ambiguity: `knowledge.context()` lists the candidate ids with tables and models before columns (§26), and the CLI prints candidates.

6.5 `_affected_neighbors` (L123) / `_upstream_neighbors` (L140)

The two generators that read `IMPACT_DIRECTION`. For a node they scan out-edges and in-edges:

- `_affected_neighbors` yields the **out**-edge target when the type is `forward`, and the **in**-edge source when the type is `reverse`.
- `_upstream_neighbors` yields exactly the opposite.

Both honour `include_inferred`. Every traversal in the library is built from these two functions, so "downstream" and "upstream" can never drift apart.

6.6 `_bfs(...)` (L233) — the traversal primitive

A breadth-first walk that records the **minimum depth** at which each node is reached, with two extras:

- `seen` is passed in, so callers can pre-seed it (`impact()` seeds it with all changed roots so a root never appears in its own blast radius).
- `column_filter`: when following a `CONTAINS` edge into a column, the column is skipped unless its name matches the filter. This implements the rename heuristic (§6.8) without a second traversal.

Depth is capped by `max_depth` before expansion, so a limit of 2 costs two rounds, not a full walk.

6.7 `upstream(node, ...)` (L157)

The mirror of `_bfs` over `_upstream_neighbors`, written as its own loop for clarity. Returns `{node_id: depth}`. `lineage()` (L180) simply calls `upstream()` and `impact()` and returns both.

6.8 `impact(changed, ...)` (L262) — blast radius, including column semantics

For each changed root:

1. BFS from the root with `seen` pre-seeded with all roots.
2. If the root is a **column** (`_column_parent`, L221, resolves the parent via `meta["parent"]` or an incoming `CONTAINS` edge), the walk continues from the parent table/model at depth 1, this time with `column_filter = <column name>`.

Step 2 is the answer to "if I rename `customer_id`, what breaks?" when the SQL that would prove column lineage is unavailable: real column→column edges are followed when they exist, and *in addition* same-named columns of downstream tables are flagged — marked `inferred` / `via: same-name column` in trees so a reviewer can see it is a guess. Results across multiple roots are merged keeping the minimum depth. Tests: `test_column_impact.py` (4 tests), `test_sql_column_lineage.py`.

6.9 `impact_tree` (L305) / `upstream_tree` (L193)

Depth-first builders that produce nested dicts (`id`, `name`, `type`, `via`, `provenance`, `children`) for rendering. A shared `seen` set prevents infinite recursion in cyclic graphs (dbt projects are acyclic; code call graphs are not) and means each node appears once, at its shallowest position. When the root is a column, the parent table is attached as a child with `via: "contains"` so the tree reads top-down.

6.10 `impact_paths(changed, target, cutoff=25)` (L296)

Builds a **direction-normalised** `DiGraph` via `_impact_digraph` (L357) — every edge is inserted in the direction impact actually flows, reversing `calls` / `imports` / `depends_on` — then delegates to `networkx.all_simple_paths`. This answers "*why* does this dashboard depend on that function?" with concrete chains. `cutoff` bounds path length so pathological graphs cannot hang the CLI.

6.11 `hotspots(top=10)` (L399)

Runs `impact()` from every non-column node and ranks by blast radius, breaking ties by total degree then id (so output is stable). Columns are excluded because they would dominate the list without being actionable. Complexity is O(V·E) — acceptable for the graph sizes this library targets (thousands of nodes) and only run on demand.

6.12 `link_table_aliases()` (L372)

The code↔data bridge's second half. Code says `analytics.fact_booking`; dbt and the warehouse say `prod.analytics.fact_booking`. For every table-like node, the method looks for another whose bare id ends with `"." + short` and has strictly more dotted parts. **Only if exactly one candidate exists** are two `depends_on` edges added (both directions, `via: "alias"`), so impact flows across the two spellings. Ambiguity is left alone rather than guessed. Returns the number of pairs linked; the CLI prints it. Test: `test_bridge_detection.py::test_alias_linking_bridges_code_and_dbt`.

6.13 Exports (L433–472)

- `to_graphml(path)` — via a plain `DiGraph` copy (networkx cannot serialise our objects), keeping `name`, `type`, `path`, `provenance` as attributes. For Gephi / yEd.
- `to_dot()` — Graphviz; inferred edges are dashed, so heuristics are visible in the picture.
- `to_cypher()` — `MERGE` statements with the node type as the Neo4j label (`dbt_model` → `DbtModel`) and provenance on the relationship. Idempotent by design, so the same export can be replayed into a live Neo4j.
- `_esc` (L518) escapes backslashes and both quote characters for DOT/Cypher string literals.

6.14 Persistence (L473–498)

`to_dict()` emits `{"version": 2, "nodes": [...], "edges": [...]}`; `save()` writes it with `indent=2, sort_keys=True` — deterministic output, so two builds of an unchanged project produce byte-identical files and `git diff` on a committed graph is meaningful. `load()` reads with `utf-8-sig` (see §7.4 on the BOM incident).

6.15 `diff_graphs(old, new)` (L500)

Set difference over node ids and `(src, dst, type)` triples, returning added/removed nodes and edges plus two convenience keys — `added_columns` / `removed_columns`. A removed column in a scheduled build is the earliest signal of a breaking schema change, which is why it is surfaced separately. Powers `datagraph graph-diff`. Test: `test_provenance_exports.py::test_diff_graphs_detects_schema_drift`.

Part II — Extractors

Thirteen extractors plus a plugin registry. Every one obeys the same contract and can be used alone.

7. `extractors/base.py` — the contract

```
class Extractor(Protocol):
    name: str
    def extract(self) -> ImpactGraph: ...
```

22 lines. Constructor takes the artifact (a path, a DSN, a connection); `extract()` returns a *fragment*. No extractor mutates a shared graph, holds global state, or knows about another extractor — which is why they can run in any order, in parallel, or individually in tests.

8. `extractors/python_extractor.py` (232 lines)

8.1 Two passes, and why

`extract()` (L31) walks `root.rglob("*.py")`, skipping `_SKIP_DIRS` (`.git` , `.venv` , `venv` , `__pycache__` , `node_modules` , `.tox` , `dist` , `build`).

Pass 1 parses every file and builds three indexes: `parsed` (rel path → AST), `module_index` (dotted module name → file id), `func_index` (bare function name → [node ids]). It emits `file:` nodes and, via `_walk_defs` (L138), `func:` / `class:` nodes with `meta["lineno"]` and `meta["end_lineno"]` , plus `CONTAINS` edges from file to definition.

Pass 2 needs those indexes, hence the split: imports and calls can only be resolved once every definition in the project is known. Files that fail to parse (`SyntaxError`) are skipped, not fatal — one bad file must not abort a repo scan.

The line numbers stored in pass 1 are what makes `git diff` map to *functions* rather than files (§12), which is the single most important detail in the whole extractor.

8.2 Imports — `_imported_modules` (L155) + `_resolve_module` (L124)

`import a.b.c` and `from a.b import d` are both collected. `_resolve_module` tries progressively shorter dotted prefixes against `module_index` , so `from etl.load import x` resolves to `file:etl/load.py` even when the scan root is the package parent. Unresolvable imports (stdlib, third-party) are dropped rather than creating phantom nodes. Provenance: `extracted` — the import statement is hard evidence.

8.3 Calls — `_calls` (L203)

Yields `(caller_qualname, callee_bare_name)` for every `ast.Call` inside a function body. Resolution is by **bare name** against `func_index` : `load_customers(conn)` links to any project function called `load_customers` . This is deliberately a heuristic — Python's dynamic dispatch makes exact resolution undecidable without type inference — so every call edge

is tagged `provenance: inferred, reason: "resolved by function name"`. `--no-inferred` removes them. Test: `test_provenance_exports.py::test_call_edges_are_inferred_and_can_be_excluded`.

8.4 SQL inside code — `_sql_strings` (L165) → the code↔data bridge

Every string constant (including f-strings and multi-line strings) inside a function or at module level is tested by `looks_like_sql` (§9). When it passes, `sql_tables()` returns the tables read and written, and the extractor emits:

- `DEPENDS_ON` from the owning function to each table read,
- `WRITES_TO` from the owning function to each table written,
- provenance `extracted` when sqlglot parsed the statement cleanly, `inferred` when only the regex fallback matched, `via: "sql-in-code"` in both cases.

This is what connects `load_customers()` to `table:fact_sales` with no configuration, and — after `link_table_aliases()` — through to the dbt model and the dashboard. Test: `test_bridge_detection.py::test_python_sql_strings_become_table_edges`.

9. `extractors/sql_in_code.py` (89 lines) — is this string SQL?

Two-stage detection, because false positives pollute the graph:

1. `looks_like_sql(text)` (L30) requires **both** a keyword (`_SQL_HINT` : select/insert/update/delete/merge/create/with/copy/truncate) **and** a shape (`_SQL_SHAPE` : `from|into|join|update|table` followed by an identifier). A sentence containing the word "select" fails the second test.
2. `sql_tables(text, dialect=None)` (L41) first replaces placeholders (`{x}` , `%s` , `%(name)s` , `:param` , `?` , `$1` — `_PLACEHOLDER_RE`) so parameterised SQL still parses, then tries **sqlglot**. On success it returns `(reads, writes, certain=True)`. On failure it falls back to regexes (`_READ_RE` for `from|join` , `_WRITE_RE` for `insert into|merge into|update|create table/view`) and returns `certain=False` , filtering out SQL keywords (`_KEYWORDS`) that would otherwise be mistaken for table names.

The `certain` flag is what the caller turns into `extracted` vs `inferred` provenance — an honest signal of whether a parser or a regex produced the edge.

10. `extractors/sql_extractor.py` (257 lines) — real SQL and column lineage

Optional dependency: `sqlglot` (`pip install datagraph-core[sql]`). `_require_sqlglot()` (L33) raises one clear message naming the extra.

10.1 `SqlExtractor.extract()` (L50)

Reads every `*.sql` file under the root (`utf-8-sig`), splits it into statements with `sqlglot.parse` , and for each `CREATE TABLE/VIEW ... AS SELECT` or `INSERT INTO ... SELECT` (`_extract_statement` , L70) emits: a `file:` node, the target `table:` / `view:` node, `DEPENDS_ON` edges to every source table (`source_tables` , L113), and column-level edges via `add_column_lineage` .

10.2 `qualify_with_schema(query, dialect, schema)` (L154)

The most subtle function in the extractor. `SELECT *` chains (staging → intermediate → mart) carry no column names in the SQL itself. Given a schema mapping (`{db: {schema: {table: {column: type}}}}`) this calls sqlglot's `qualify` to expand stars through CTEs and subqueries, returning the qualified query and its real output columns. The schema comes from dbt's `catalog.json`, the warehouse, or the manifest's declared columns. Without it, a `select *` model produces zero column edges — which is exactly the bug the real jaffle_shop fixture exposed during development (§39).

10.3 `column_lineage(query, dialect, schema)` (L173)

For each output column, walks sqlglot's lineage tree down to the **leaves** — the base `(table, column)` pairs — through aliases, CTEs and renames. `total_order_amount` in a mart resolves to `stg_payments.amount`, not to the intermediate alias.

10.4 `add_column_lineage(...)` (L211)

Turns that mapping into graph structure: `column:` nodes for the target, `CONTAINS` from the target relation, and `DEPENDS_ON` from target column to each source column. `resolve_relation` is a callback that lets the dbt extractor map a physical relation name back to `dbt:<model>` so column edges connect models rather than tables. Provenance `extracted` — a parse tree is evidence. Tests: `test_sql_column_lineage.py`, `test_dbt_compiled_lineage.py`, `test_jaffle_shop.py`.

11. `extractors/dbt_extractor.py` (322 lines) — the richest source

`DbtExtractor(manifest_path, column_lineage=True, dialect=None, catalog_path=None)` (L38). `catalog.json` is auto-detected next to the manifest.

`extract()` (L54) processes the manifest in a fixed order:

1. **Tests index.** All `resource_type == "test"` nodes are grouped by the model they depend on, so a model node can carry `meta["tests"] = ["unique_customer_id", ...]`.
2. **Models / seeds / snapshots.** For each: a `dbt:<name>` node with `unique_id`, `schema`, `database`, `materialized`, `description`, `tags`, `owner`, `sql` (compiled code, truncated to 4 000 chars) and `tests`; `depends_on.nodes` becomes `DEPENDS_ON` edges; the source file becomes a `file:` node with `CONTAINS`; the physical relation becomes a `table:` / `view:` node with `WRITES_TO` (so "the model" and "the table it materialises" stay distinct but linked).
3. **Columns.** Declared columns from the manifest plus catalog-only columns, each with `data_type`.
4. **Sources** → `source:<source_name>.<name>` with owner and columns.
5. **Exposures** → `exposure:<name>` typed by `_EXPOSURE_TYPES` (dashboard → DASHBOARD, notebook/ analysis/ml → REPORT), owner from `owner.name` / `owner.email`, `EXPOSES` edges from every model it depends on.
6. **Column lineage** (`_add_column_lineage`, L253) — parses each model's `compiled_code` with the schema mapping from `_schema_mapping` (L204) and emits column→column edges through `add_column_lineage`, mapping relation names back to `dbt:` ids.

`_owner_of` (L294) looks in `meta.owner`, `config.meta.owner`, then `group`, then `meta.team` — teams label ownership differently and the impact report is only useful if it finds the owner. Anything sqlglot cannot parse is collected in `self.unparsed` and written next to the graph as `<graph>.unparsed.json` — the input for the optional LLM fallback (§30). Tests: `test_dbt_extractor.py` (8), `test_dbt_compiled_lineage.py` (4), `test_jaffle_shop.py` (6, on dbt's real public project).

12. `extractors/git_extractor.py` (119 lines) — diff → functions

12.1 `collect_changes(repo, base="HEAD", head=None)` (L35)

Runs `git diff --unified=0 <base>` (or `<base>...<head>`) plus `--name-only`, through `_run_git` (L25) which pins `text=True, encoding="utf-8", errors="replace"` — added after a real crash on a non-UTF-8 diff on Windows. `_HUNK_RE` (L13) parses `@@ -a,b +c,d @@` headers into **new-file line ranges**. Returns a `ChangeSet(files, ranges)`.

`base="HEAD"` means "my uncommitted work"; `base="origin/main", head="feature/x"` is the PR case.

12.2 `changed_node_ids(graph, changes)` (L70)

Maps a `ChangeSet` onto ids:

- every changed file → its `file:` node (`_match_paths`, L99, tolerates path prefixes so a repo-root diff matches a graph built with `--repo src`);
- every function/class whose `[lineno, end_lineno]` **intersects** a changed range (`_spans_intersect`, L110);
- any node *contained* in a changed file (a dbt model whose `.sql` file changed).

The intersection test is why editing one function in a 500-line file does not implicate the other functions — visible in the impactgraph tour where `load_products()` stays out of the blast radius. Tests: `test_git_extractor.py` (2), `impactgraph/test_check.py`.

13. `extractors/warehouse_extractor.py` (255 lines) — any database

13.1 `connect(dsn)` (L41)

Accepts, in order: a `.db`/`.sqlite` path or `sqlite:///…` → `sqlite3`; `duckdb://…` or a `.duckdb` file → `duckdb`; anything else → SQLAlchemy `create_engine(dsn).raw_connection()`. If SQLAlchemy is missing, the error names the driver to install. A DB-API connection can also be passed directly, which is how the Snowflake key-pair example reuses an existing session.

13.2 Two code paths

`_is_sqlite()` (L117) selects `_extract_sqlite()` (L178), which uses `sqlite_master` and `PRAGMA table_info` / `PRAGMA foreign_key_list` (SQLite has no `information_schema`). Everything else goes through `extract()` (L121) and standard `information_schema.tables` / `.columns`.

13.3 `_where()` (L89) — filtering, and a bug worth documenting

Builds the WHERE clause: `database` → `lower(table_catalog) = '<db>'`, `schemas` → `lower(table_schema) IN (...)`. When no filter is given it excludes engine-owned objects:

```
SYSTEM_CATALOGS = ("system", "temp")                    # DuckDB
SYSTEM_SCHEMAS  = ("information_schema", "pg_catalog", "pg_toast", "sys", "sysibm",
                   "syscat", "mysql", "performance_schema", "innodb", "temp", "pg_temp_1")
```

Before 0.8.4 only `information_schema` and `pg_catalog` were excluded. On **MySQL**, `table_schema` *is* the database, so `--warehouse mysql://…` ingested `mysql`, `sys` and `performance_schema` — hundreds of junk tables; on **DuckDB** its internal `duckdb_*` / `sqlite_*` views appeared. Found while writing the DuckDB section of the example tour, fixed in 0.8.4. All literals go through `escape_literal` (§28). Tests: `test_warehouse_sqlite.py::test_system_catalogs_and_schemas_are_excluded`, `::test_duckdb_schema_has_no_system_objects`.

13.4 Foreign keys and views

`_add_foreign_keys` (L151) joins `referential_constraints` with `key_column_usage` twice (child and parent side, matched on `ordinal_position` for composite keys) and emits, via `_add_fk` (L229), a column→column `DEPENDS_ON` **and** a table→table `DEPENDS_ON`, both `via: "foreign_key"`. The whole query is wrapped in `try/except` because Snowflake and BigQuery do not expose FK metadata — an engine without foreign keys must yield fewer edges, not an error.

`_add_view_lineage` (L169) reads `view_definition` and parses it with `_view_lineage` (L237) → sqlglot → source tables and column lineage, `via: "view_definition"`. Also `try/except`: engines that hide view SQL degrade to tables and columns only.

14. `extractors/openlineage_extractor.py` (131 lines)

Reads a JSON array or NDJSON of OpenLineage run events (`_read_events`, L112 — the format Airflow, Spark and Marquez emit). Per event (`_add_event`, L41): each input/output dataset becomes a `table:` node (namespace kept in meta), the job becomes `job:<namespace>/<name>` (typed DAG); the job `DEPENDS_ON` its inputs and `WRITES_TO` its outputs, and **each output** `DEPENDS_ON` **each input** so dataset-level lineage exists even without column facets. The `schema` facet creates columns; the `columnLineage` facet creates column→column edges; the `ownership` facet sets the dataset owner. This is how runtime-observed lineage joins the static graph. Tests: `test_openlineage.py`.

15. `extractors/lineage_file_extractor.py` (139 lines)

DataHub's `datahub-lineage-file` YAML/JSON shape plus a documented superset: per entity an `upstream` list, an optional `owner`, an optional `columns` map, and DataHub-style `fineGrainedLineages`. `_entity_id` (L100) maps `{name, type, platform, env}` to a `table:` id; `_split_col` (L132) resolves `analytics.dim_customer.customer_key` into (table id, column). YAML needs PyYAML; JSON always works. Curated lineage that a team already maintains for DataHub therefore drops straight in. Tests: `test_lineage_file.py` (3).

16. `extractors/datahub_extractor.py` (163 lines)

Live import over GraphQL. `DataHubExtractor(server, token=None, query="*", max_entities=2000, page_size=100, transport=None)` (L53). `_http` (L69) posts `_SEARCH_QUERY` with `Authorization: Bearer <token>` (token from the argument or `$DATAHUB_TOKEN`), paginating until `max_entities`. `_dataset` (L94) converts each result: URN → id via `_urn_to_id` (L146, regex `_URN_RE` extracts the dataset name from `urn:li:dataset:(urn:li:dataPlatform:snowflake,db.schema.table,PROD)`), schema fields → columns, `upstreamLineage` → `DEPENDS_ON`, `fineGrainedLineages` → column edges, ownership → owner. The `transport` parameter is a seam for tests — `test_datahub.py` runs the full path against a stub, so no network is needed in CI.

17. `extractors/airflow_extractor.py` (224 lines)

AST-based; **Airflow is never imported**, so the extractor works on a laptop with no scheduler. `_find_dag_ids` (L150) finds `DAG("id")` / `with DAG(...)` / `@dag`. Operators become `task:<dag>/<id>` nodes (`_task_id_of`, L174). Dependencies come from three syntaxes: `a >> b >> c` and `a << b` (`_flatten_chain`, L134, which also normalises the direction), list forms `[a, b] >> c`, and `chain(...)`. `python_callable=fn` (`_callee_name`, L191) becomes a `DEPENDS_ON` edge to the function node, so a change to that function reaches the task. String arguments that look like SQL are run through `sql_tables` (§9) so `SnowflakeOperator(sql="INSERT INTO …")` yields table edges. Tests: `test_airflow.py` (3).

18. `extractors/lambda_extractor.py` (216 lines)

Handles both **Serverless Framework** (`_serverless`, L48) and **AWS SAM / CloudFormation** (`_sam`, L64). Each function becomes `lambda:<name>`; `_handler_func_id` (L96) converts `etl/load_customers.load_customers` into `func:etl/load_customers.py::load_customers`, linking the lambda to the Python node the Python extractor created. `_event` (L110) turns triggers into structure: `http` → `api:GET /path`, plus S3/SQS/DynamoDB sources. `_env_refs` (L135) reads environment variables that name tables (`SALES_TABLE: fact_sales`) and links them; `_ref_name` (L163) resolves `{Ref: X}`, `{Fn::GetAtt: [X, Arn]}`, `!Ref X` and `${self:...}`. YAML via PyYAML; `_load` (L190) raises the "install datagraph[yaml]" message when missing. Tests: `test_lambda.py` (4).

19. `extractors/js_extractor.py` (147 lines)

Regex-based and explicitly best-effort (a JS parser would be a heavy dependency for a secondary language). `_FUNC_RE`, `_ARROW_RE`, `_METHOD_RE` find declarations, arrow functions and class methods; `_functions` (L100) assigns each a start line and an end line of "next start − 1", which is enough for diff intersection. `_IMPORT_RE` covers `import … from`, `export … from` and `require()`; `_resolve_import` (L124) resolves relative specifiers against the file set, trying `.js/.ts/.tsx/…` and `/index.*`. Calls are matched by `_CALL_RE` minus `_JS_KEYWORDS` (`if`, `for`, `switch`, …) and tagged `inferred`. Template literals and quoted strings are scanned for SQL (§9). Tests: `test_js.py`.

20. `extractors/registry.py` (95 lines) — plugins

```
@dataclass
class ExtractorPlugin:
    name: str
    factory: Callable[..., object]      # a class or function returning something with .extract()
    help: str = ""
    value_name: str = "PATH"
    options: Dict[str, str] = {}        # extra CLI flags: --<name>-<option>
    source: str = "registered"
```

`ExtractorPlugin.extract(value, **options)` (L32) calls the factory, calls `.extract()`, and raises `TypeError` if the result is not an `ImpactGraph` — a plugin cannot corrupt the graph by returning something else. `register()` (L44) adds one programmatically; `load_entry_points()` (L54) discovers packages that declare the `datagraph.extractors` entry-point group, is idempotent (`_LOADED`), and wraps a failing import in `_broken` (L90) so one bad plugin cannot break the CLI. `plugins()` (L80) and `get()` (L85) trigger discovery lazily.

The CLI (§33) turns every plugin into `datagraph build --<name> VALUE [--<name>-<option> V]` automatically. Tests: `test_plugins.py` (3).

Part III — Analysis, outputs and services

21. `analysis/impact.py` (86 lines) — one call, four answers

```
@dataclass
class ImpactAnalysis:
    changed: List[str]              # resolved root ids
    affected: Dict[str, int]        # node id -> minimum depth
    risk: Dict                      # {"score": float, "level": str}
    recommended_tests: List[str]
    trees: List[Dict]               # one impact_tree per root
    owners: Dict[str, List[str]]    # owner -> affected asset names
    include_inferred: bool
    _graph: Optional[ImpactGraph]   # kept for summary_by_type(), excluded from JSON
```

`analyze_impact(graph, changed, max_depth=None, include_inferred=True)` (L47):

1. **Resolve** each reference through `graph.resolve()`, falling back to the raw id if it already exists; unresolvable references are dropped (the CLI reports them).
2. `affected = graph.impact(resolved, ...)`.
3. `risk = risk_score(graph, affected)` (§22).
4. `tests = recommend_tests(graph, affected)` (§23).
5. `trees = [graph.impact_tree(nid, ...) for nid in resolved]`.
6. **Owners**: for every affected node with `node.owner`, group asset names under the owner and sort — this is the "who do I tell" list.

`summary_by_type()` (L24) counts affected nodes per type; `to_dict()` (L34) returns a JSON-safe dict (no `_graph`) — the exact payload the CLI's `--json`, the MCP tools and impactgraph's Markdown all use. Test: `test_graph.py::test_analysis_to_dict_is_json_serializable`.

22. `analysis/risk.py` (71 lines) — deterministic scoring

```
score = Σ over affected nodes ( weight(type) × profile_factor × depth_discount ) + breadth_bonus
```

- **`NODE_WEIGHTS` (L10):** dashboard/report/exposure 8 · api 6 · lambda 4 · table/view/dbt_model/ snapshot 3 · dag 3 · seed/source/task 2 · function/class/column/module/file 1. Rationale: a broken dashboard is seen by the business; a changed helper function is not.
- **`profile_factor`:** if the node carries a profile (§25), `row_count == 0` → ×0.5 (an empty table is low stakes), `row_count ≥ 1,000,000` → ×1.5. This is why profiling makes risk *data-aware*.
- **`depth_discount = max(0.5, 1.0 - 0.1 × (depth - 1))`** — direct hits count fully, and the floor of 0.5 stops a dashboard six hops away from vanishing. Distance reduces confidence, not consequence.

- `breadth_bonus = min(direct_hits, 10)` — many things touching a node at depth 1 is itself a signal, capped so a fan-out of 200 cannot swamp the type weights.
- **Thresholds (L60):** ≥40 CRITICAL, ≥18 HIGH, ≥6 MEDIUM, else LOW. Score rounded to one decimal.

No randomness, no model, no configuration file: the same graph always yields the same number, which is what makes `--fail-on` safe to put in CI.

23. `analysis/tests_recommender.py` (56 lines)

`recommend_tests(graph, affected)` (L10) maps each affected node type to a concrete action, de-duped via a `seen` set while preserving order:

Affected type	Recommendation
column	"Add/verify a dbt relationship + not_null test for column 'x' (parent)"
table / view	"Run a schema/contract check on X (column names & types)"
function	"Run unit tests covering `path` (function 'x')"
api	"Run the API contract test for 'x'"
lambda	"Run an integration test invoking lambda 'x'"
dashboard / report / exposure	"Manually validate 'x' after deploy (numbers & filters)"
dag	"Trigger a test run of DAG 'x' in a non-prod environment"
task	"Run task 'x' of DAG 'y' against a non-prod target"

dbt models are collected and emitted as **one** command — `dbt build --select modelA+ modelB+` (first five, sorted) — because that is what a person would actually run. If nothing is affected the plan is "No downstream artifacts detected; run the standard test suite."

24. `analysis/relationships.py` (65 lines) — the schema map

`relationships(graph, search=None, include_columns=True)` (L12) returns:

```
{"tables": [ {id, name, type, owner, profile, columns[], depends_on[], dependents[]} ],
 "table_relationships":  [ {source, target, via, provenance} ],
 "column_relationships": [ {from, to, via, provenance} ]}
```

One pass over `graph.edges()` classifies each edge: `CONTAINS` from a table-like node to a column fills `columns` (with `data_type`, `primary_key` and `profile`); `DEPENDS_ON` / `WRITES_TO` between two table-like nodes fills `table_relationships` and the per-table `depends_on` / `dependents`; column→column edges fill `column_relationships`. `via` prefers `edge.meta["via"]` (`foreign_key`, `view_definition`, `alias`, `sql-in-code`, `llm`) and falls back to the edge type, so the reader always knows *why* two tables are related. `search` filters tables by substring — the practical entry point on a warehouse with thousands of relations.

25. `profiling.py` (203 lines) — facts about the data

`profile_warehouse(connection, graph, tables=None, sample=100_000, top_values=True, max_columns=60, log=None)` (L28). A DSN string is accepted and opened via `connect()`.

Target selection (`_targets`, L118): explicit names (resolved through the graph) or every `TABLE`/`VIEW` node whose `meta["source"]` is `warehouse`/`sqlite` — profiling never touches a node that did not come from a database.

Per table: 1. `SELECT COUNT(*) FROM <relation>` → `row_count`. A failure here (permission denied, dropped table) is caught, stored as `profile["error"]`, and the loop continues — one unreadable table must not abort a 500-table run. 2. **One** aggregate query for all columns: `SELECT COUNT(*), COUNT(c1), COUNT(DISTINCT c1), MIN(c1), MAX(c1), COUNT(c2), … FROM (SELECT * FROM <rel> LIMIT <sample>) t`. One round trip instead of 4 × N, which is what makes profiling usable on a remote warehouse. `null_pct` is derived as `(sampled_rows - COUNT(col)) / sampled_rows × 100`. 3. **Top values** (optional): `SELECT c, COUNT(*) FROM (SELECT c FROM <rel> LIMIT n) GROUP BY c ORDER BY COUNT(*) DESC LIMIT 5`, for the first 20 columns only. 4. **Freshness**: the maximum `max` over columns that look temporal (`_looks_temporal`, L169 — name hints `date`, `time`, `timestamp`, `_at`, `_ts`, `day`).

Privacy. `is_sensitive_column(col.name)` (§28) decides masking: counts (`null_pct`, `distinct`, `sampled_rows`) are kept, but `min`/`max` are set to `None`, `top_values` is skipped entirely, and `masked: True` is recorded. Personal data never enters the graph, the wiki, an HTML file or an LLM prompt. Test: `test_security.py::test_profiling_masks_sensitive_values`.

Identifiers are quoted with `_q` (L139, `"` doubled); `_relation_sql` (L132) rebuilds the relation name per engine (bare name for SQLite; catalog.schema.name elsewhere, quoting only the parts that need it). `_jsonable` (L175) converts dates/Decimals so the graph stays JSON-serialisable. `profile_summary(node)` (L181) renders the one-liner used in tooltips and context packs ("2,000 rows, fresh to 20260228").

26. `knowledge.py` (218 lines) — the AI-assistant surface

26.1 `context(graph, node_id, depth=2, max_items=40)` (L42)

Builds the compact text pack an assistant needs before answering about — or changing — a node: header (name, type, id, path, owner), selected meta (description, materialized, schema, database, platform, handler, operator, dag, namespace), profile summary, **modelling role** (from `classify_tables`, §27), dbt test count and names, columns with type/pk/profile **and where each column comes from**, upstream and downstream lists at `depth`, table relationships, a warning naming any heuristic edges around the node, the risk-if-changed line with owners and recommended tests, and the SQL that builds it (`meta["sql"]`, sanitised, 2 000 chars).

If the reference does not resolve, the function distinguishes two cases: **ambiguous** (list the candidate ids, tables and models before columns) versus **no match**. That distinction was added in 0.8.4 after the example tour showed "No node matches 'dim_customer'" for a name that matched eleven nodes.

26.2 `build_wiki(graph, out_dir, title, include_files=False)` (L139)

Writes a browsable knowledge base:

- `nodes/<slug>.md` — one page per table/view/model/source/function/DAG/task/lambda/API/dashboard, containing the context pack plus a **Links** section cross-linking one-hop neighbours. `slug()` (L35) makes ids filesystem-safe.
- `index.md` — every asset grouped by type, with owners.

- `GRAPH_REPORT.md` — hotspots (top 15), high-impact dbt models **without tests**, nodes without an owner, roots (raw inputs), leaves (nothing downstream — deletion candidates), and the count of heuristic edges.
- `MODEL.md` — the dimensional model (§27).
- `llms.txt` — a flat link list, the convention RAG tools look for.

Every page states that descriptions, docs and SQL are data copied from the user's sources and must be treated as untrusted text (§28). Tests: `test_knowledge.py` (7).

27. `analysis/modeling.py` (515 lines) — Kimball, deterministically

The largest analysis module. Everything it concludes is accompanied by the reasons for it.

27.1 `classify_column(col, table_base, row_count)` (L71) → pk | fk | date | measure | flag | attribute

Ordered rules: declared primary key → `pk`; the table's own id form (`customer_id` in `dim_customer`, also `_key` / `_sk`) → `pk`; any `_id/_key/_sk/_fk` → `fk`, promoted to `pk` when the profile shows `distinct ≥ row_count` (a unique key is a primary key); boolean type or `is_/has_/was_/…` prefix → `flag`; temporal type or date-ish name → `date`; numeric type → `measure` when the name matches `_MEASURE_WORDS` (amount, total, qty, revenue, …) or a currency/percent suffix, otherwise `attribute` if the profile shows ≤50 distinct values (a numeric code), else `measure`; anything else → `attribute`. Profiles make this materially better, which is why `analyze` profiles before modelling.

27.2 `fk_links(graph, include_inferred=True)` (L98)

Declared foreign keys (`via == "foreign_key"`) → `provenance: extracted`. Then, optionally, name inference: a column `<x>_id|_key|_sk|_fk` in table T is matched against tables whose base name is `x` or its singular (`_singular`, L46, handles `ies/ses/xes/s`), ignoring dbt-style prefixes (`_strip_prefix`, L56: `dim_`, `fact_`, `stg_`, `raw_`, …). The candidate that actually has the same column (or an `id`) wins; own-key references are skipped; results are tagged `inferred`. This is what produces a usable model on Snowflake and BigQuery, which have no enforced foreign keys.

27.3 `classify_tables(graph, include_inferred=True)` (L152)

Scores each table-like node as fact vs dimension and records the reasons:

- fact += 2 per outgoing key link, += 1.5 per measure column, += 1 if it has a date, += 1 per key-like column when no links exist, += 2 if the name matches `_FACT_NAMES`;
- dimension += 2 per incoming reference, += 0.75 per attribute, += 1 for a primary key, += 2 if the name matches `_DIM_NAMES`;
- **bridge** when ≥2 outgoing links (or ≥2 fk columns) and no measures/attributes/dates;
- **derived** for a view with no key links — a report-layer object, not a base fact (added 0.7);
- **unknown** when no columns are known (e.g. a table seen only through SQL), with that stated as the reason;
- confidence = normalised margin between the two scores, floor 0.4.

27.4 `star_schema(graph, include_inferred=True)` (L233)

Assembles the model and, per fact, the **Kimball four steps**: business process (from the name), grain (date/date-key first, then the dimension keys), dimensions, fact measures — plus `additivity` (measures named balance/inventory/stock/level/headcount/snapshot are marked *semi-additive*). Dimensions get key, attributes, used-by,

snowflake links, and an **SCD verdict**: type 2 when `valid_from` / `valid_to` / `is_current`-style columns exist, type 1 when only `updated_at` exists, `static` for date/calendar dimensions, otherwise "undecided" with a recommendation. A **bus matrix** (`{fact: [dimensions]}`) and the list of **conformed dimensions** (used by ≥2 facts) complete it.

The `issues` list is the actionable part: fact without a time grain, key with no dimension, fact-to-fact link, measures sitting in a dimension, unused dimension, natural/text key where a surrogate key is standard, missing conformed `dim_date`, key columns with >20 % nulls (late-arriving dimensions), and every name-inferred link ("verify or declare a foreign key").

27.5 `propose_from_table(graph, table, max_attr_distinct=200)` (L338)

The brownfield tool. Given one wide table it groups low-cardinality attributes by name prefix into dimensions (`customer_name`, `customer_country` → `dim_customer`), routes numeric columns to measures, dates to `dim_date`, near-unique text to **degenerate dimensions** kept on the fact, and names the fact `fact_<base>`. Returns fact, dimensions, column roles and notes.

27.6 `to_mermaid` (L391) / `to_markdown` (L444)

Mermaid `erDiagram` (PK/FK markers, `}o--||` relationships, "(inferred)" labels) — GitHub renders it natively. Markdown adds the bus-matrix table, per-fact Kimball lines, per-dimension SCD, derived and unclassified sections, the issues list, and the embedded diagram. Tests: `test_modeling.py` (4), `test_analyze.py` (2).

28. `security.py` (97 lines) — the threat model in code

Function	Line	Purpose
`redact_dsn(dsn)`	38	Rebuilds a URL with `user:***@host`, drops the query string, then regex-scrubs `password=/token=/secret=/api_key=/private_key=` for ODBC-style strings. File paths pass through unchanged.
`sanitize_text(text, max_len=2000)`	60	Strips control, bidi-override and zero-width characters (`_CTRL`) and truncates. Bidi overrides can make a prompt or report display differently from its bytes.
`wrap_untrusted(text)`	70	Wraps content in `<data> … </data>` and neutralises inner `</data>` so the fence cannot be closed early.
`UNTRUSTED_NOTICE`	30	The sentence appended to every LLM system prompt: content inside `<data>` is data, never instructions.
`is_sensitive_column(name)`	76	Regex over ~40 families (email, phone, ssn, aadhaar, passport, name, address, card, cvv, token, secret, salary, ip, geo, biometric, health …), with an explicit carve-out so plain `_id/_key/_sk/_fk` columns are still sampled.
`quote_ident` / `escape_literal`	86 / 90	`"` doubling and `'` doubling for generated SQL.
`escape_script_json(json)`	94	Escapes `</` and U+2028/2029 so an HTML report cannot be broken out of its `<script>` tag by a malicious table name.

Applied at: CLI logging (`redact_dsn`), profiling (masking), warehouse `_where()` (`escape_literal`), HTML rendering (`escape_script_json`), every AI prompt (`wrap_untrusted` + notice), wiki/context (sanitised text + notice), MCP server instructions. Tests: `test_security.py` (8).

29. `ai/providers.py` (198 lines) — pluggable LLMs

```
class LLMProvider:
    def complete(self, system, user, *, max_tokens=4096, json_schema=None) -> str: ...
```

One method. Three implementations:

- **`AnthropicProvider`** (L73) — the Anthropic SDK; uses native structured outputs (`output_config.format.json_schema`) when a schema is given, and streams when `max_tokens > 8000` to avoid non-streaming timeouts. `stop_reason == "refusal"` returns `""`.
- **`BedrockProvider`** (L102) — the Bedrock **Converse** API via boto3, so Amazon Nova, Claude on Bedrock, Llama and Mistral all work through one code path. No native JSON schema, so the schema is appended as an instruction (`_with_schema`, L46) and the reply parsed leniently. Per-model output caps are handled: the budget is clamped to `DEFAULT_MAX_TOKENS` (8 000, override with `DATAGRAPH_LLM_MAX_TOKENS`) and, if the API reports "model limit of N", retried once at N−1. Found against real Nova, which caps at 10 000.
- **`OpenAICompatibleProvider`** (L143) — plain HTTPS POST to `/v1/chat/completions` with `urllib.request`; **no dependency at all**. Works with OpenAI, Azure OpenAI, Ollama, vLLM, LM Studio and Groq via `DATAGRAPH_LLM_BASE_URL`. A `transport` seam makes it testable offline.

`extract_json(text)` (L52) parses a reply that may be fenced or wrapped in prose: try raw JSON, then the contents of a `` ``` `` fence, then the widest `span.` get_provider(provider, model, api_key) `(L184) accepts an instance (returned as-is), a name, or` None `→` $DATAGRAPH_LLM_PROVIDER `→ Anthropic; the model falls back to` $DATAGRAPH_LLM_MODEL `then the provider default. **Credentials are only ever read from the environment or the cloud SDK's own chain.** Tests:` test_providers.py` (8, all offline).

30. `ai/explain.py` (53) and `ai/lineage.py` (163)

`explain_impact(analysis, model=None, api_key=None, max_tokens=16000, provider=None)` sends the **already-computed** `analysis.to_dict()` wrapped in `<data>` tags with a system prompt that forbids inventing nodes. It returns prose; it cannot change the graph.

`suggest_lineage(graph, unparsed_sql=None, ...)` (L93) sends `schema_summary(graph)` (L61 — tables, columns with types and, when profiled, distinct/null/min/max, plus the relationships already known) together with up to 50 sanitised snippets of SQL the parsers could not read, and requests `SUGGESTION_SCHEMA` (L35): `{kind, source, target, confidence, reason}[]`.

`apply_suggestions(graph, suggestions, min_confidence=0.6)` (L127) is the gatekeeper: a suggestion is rejected unless the confidence clears the bar and **both endpoints already exist** (a column may be created only under an existing table). Accepted edges are `DEPENDS_ON` with `provenance: llm`, the confidence and the reason recorded, and are dropped by `--no-inferred`. The model can therefore fill gaps but never invent a table. Tests: `test_llm_lineage.py` (6), `test_ai_layer.py` (3).

31. `report.py` (163) and `html_report.py` (241)

Terminal. `render_analysis` (L62) prints a risk panel styled by level (`_RISK_STYLES`), the impact trees as `rich.Tree` (`_to_rich_tree`, L150 — icons per type, `via` and a `(provenance)` marker for heuristics), the affected-by-type summary, owners and the test plan. On a console that cannot encode the glyphs (Windows cp1252) `_ASCII_ICONS`

replaces them (`[dbt]` , `f()` , `[tbl]`) instead of raising — a real crash that occurred on the first Windows run. Node names go through `rich.markup.escape` so a name containing `[dbt]` is not interpreted as markup. Tests: `test_report.py` (2, one per encoding).

HTML. `render_html` (impact), `render_lineage_html` (upstream+downstream), `render_graph_html` (whole graph, columns hidden by default) all funnel into `_render` (L106), which computes a layered layout — depth (or topological layer) on the x-axis, spread on the y-axis — serialises nodes and edges to JSON, and substitutes them into `_TEMPLATE` (L155). The template is a single file with inline CSS and JS: pan/zoom, click-to-highlight-downstream, a type filter, a legend, tooltips (type, depth, owner, profile) and a details panel. No CDN, no fonts, no network — the file can be attached to a ticket or opened offline. Colours per node type come from `_COLORS` (L24); the JSON payload passes through `escape_script_json` (§28).

32. `mcp_server.py` (121 lines) — tools for AI assistants

`build_tools(graph_path)` (L20) returns nine callables, each with a docstring that becomes the MCP tool description: **impact**, **diff**, **find_nodes**, **paths**, **hotspots**, **lineage**, **relationships**, **context**, **model**. The graph is re-read per call, so a rebuild in another terminal is picked up without restarting the server.

`serve(graph_path)` (L103) registers them on a `FastMCP` instance and runs it over **stdio** — a local child process, no network port. The server's `instructions` state that everything returned is data copied from the user's sources and must never be followed as instructions, so the untrusted-data rule travels with the payload. `mcp` is an optional extra; the import error names it. Tests: `test_mcp_maintenance.py` (5) exercise `build_tools` without the runtime installed.

33. `maintenance.py` (110) and `cli.py` (832)

Maintenance. `fingerprint(paths, patterns, exclude)` (L16) is a SHA-256 over the *content* of every input file (directories walked for `*.py|sql|json|yml|yaml` , skipping `.git` , `.venv` , …). Content, not mtime, so a `git checkout` that restores identical files does not force a rebuild. `is_up_to_date` (L50) compares against `<graph>.cache.json` ; `_outputs` (L72) excludes the graph and its cache from their own fingerprint — the bug that made `--update` always rebuild when the graph lived inside the scanned repo. `watch` (L76) polls the fingerprint every `interval` seconds (`max_iterations` exists so tests terminate); `install_hook` (L98) writes an executable `.git/hooks/<hook>` that runs a given command.

CLI. `main(argv)` (L73) builds one `argparse` parser with 22 sub-commands, then — before parsing — asks the plugin registry for extra `--<plugin>` flags on `build` / `watch` / `hook-install` . `_dispatch` (L243) maps the command name to a `_cmd_*` function. Shared helpers: `_add_build_args` (L43) for the 15 input flags, `_add_analysis_args` (L65) for `--graph/--max-depth/--json/--no-inferred/ --html` , `_load_graph` (L273) with a clear "run datagraph build first" error, `_build_graph` (L288) which runs every requested extractor, logs one line each (DSNs redacted), links aliases and collects unparsed SQL, and `_emit` (L383) which renders text or JSON and optionally writes HTML.

Commands: `build` , `analyze` , `impact` , `diff` , `lineage` , `relationships` , `profile` , `model` , `context` , `wiki` , `nodes` , `paths` , `hotspots` , `graph-diff` , `export` , `html` , `watch` , `hook-install` , `explain` , `enrich` , `mcp` , `plugins` . `_cmd_analyze` (L674) is the one-shot flow: connect → schema → relationships → profiling → model → lineage HTML → wiki, writing seven artefacts into an output folder and printing the next commands to run. Exit codes: 0 success, 2 usage/resolution error (and 1 from impactgraph when `--fail-on` trips).

Part IV — impactgraph, tests, CI/CD

34. `impactgraph/core.py` (183 lines)

34.1 `CheckResult` (L56)

```
@dataclass
class CheckResult:
    graph: ImpactGraph
    changed_files: List[str]
    changed_ids: List[str]
    analysis: Optional[ImpactAnalysis]   # None when nothing mapped
    graph_path: Optional[str]
    notes: List[str]
```

`level` (L65) and `score` (L69) are properties that fall back to `"LOW"` / `0.0` when there is no analysis, so a docs-only PR never crashes a report. `breaches(fail_on)` (L72) compares against `LEVELS`, treating `None` / `"NONE"` as "never fail". `to_dict()` (L77) merges the analysis payload with `changed_files` and `notes`.

34.2 `build_graph(code, inputs, output, update, quiet)` (L91)

Deliberately calls **datagraph's own CLI** (`datagraph.cli.main`) with an argv list rather than importing extractors directly, mapping `BUILD_FLAGS` (L39) — `dbt_manifest → --dbt-manifest`, `lambda_ → --lambda`, … — onto flags. Consequence: every datagraph input works in impactgraph the day it is added, with no wrapper to maintain. Stdout is captured (`contextlib.redirect_stdout`) so a JSON or Markdown report is never polluted by build logs; a non-zero exit becomes a `RuntimeError` carrying the captured output.

34.3 `check(...)` (L112)

Graph source in priority order: a passed `graph` object → `graph_path` file → build from `code` (default: the repo) + `inputs`. Then `collect_changes(repo, base, head)` and `changed_node_ids` (§12), and finally `analyze_impact`. Two early returns with explanatory notes: "no changes detected" and "changed files do not map to any graph node (docs/config only?)".

34.4 `to_markdown(result, title)` (L155)

The PR comment: a risk-coloured heading (🟢 🟡 🟠 🔴, ✅ for no impact), the changed files, the affected summary by type, **Notify** owners, the blast radius inside a `<details>` block (so a large tree does not swamp the conversation), a `- [ ]` test checklist, and a footer noting that heuristic edges are included. `_tree_md` (L144) is depth-first with a 200-line cap.

34.5 `safe_console_text(text, stream=None)` (L24)

Emoji are correct for GitHub but fatal on a legacy Windows console: `impactgraph pr` raised `UnicodeEncodeError` on cp1252. This helper tries `text.encode(stream.encoding)`; on failure it substitutes `_ASCII_FALLBACK` (🟢 → `[LOW]`, 🟡 → `[MEDIUM]`, 🟠 → `[HIGH]`, 🔴 → `[CRITICAL]`, ✅ → `[OK]`, ⚠→ `[!]`, →/—/–/·/… → ASCII) and re-encodes with `errors="replace"`. **Files keep the emoji** (always written UTF-8); only terminal output is downgraded. Fixed in 0.7.3. Tests: `test_check.py::test_markdown_never_crashes_a_legacy_console`, `::test_output_file_keeps_utf8`.

35. `impactgraph/cli.py` (117 lines)

`main(argv)` (L100) inspects `argv[0]`: if it is not `check` / `pr` / `-h` / `--help` / `--version`, the whole argv is forwarded to `datagraph.cli.main`. So `impactgraph lineage X`, `impactgraph context Y` and `impactgraph mcp` work without impactgraph implementing them.

`_run_check` (L55) wraps `check()` in a try/except that prints `error: …` to stderr and returns 2, writes the optional HTML, renders text (rich) / JSON / Markdown, writes to `-o` as UTF-8 or prints through `safe_console_text`, and finally returns **1** when `result.breaches(--fail-on)` — the exit code CI gates on.

36. `action.yml` (113 lines) — the GitHub Action

A composite action: install `impactgraph[sql,yaml]` → `git fetch` the base ref → run `check` twice (JSON to `impact.json`, Markdown to `comment.md`) → export `level` as a step output and append the Markdown to `$GITHUB_STEP_SUMMARY` → `gh pr comment --body-file comment.md` → compare the level against `fail-on` and exit 1 if it is at or above. Inputs: `repo-path`, `dbt-manifest`, `dbt-catalog`, `sql-dir`, `airflow`, `lambda`, `js`, `openlineage`, `lineage-file`, `base-ref`, `fail-on`, `no-inferred`, `github-token`. Only permission needed: `pull-requests: write` with the default `GITHUB_TOKEN`.

37. The test suite — 145 + 6 tests, all offline

Both suites run with no network, no API key and no database server: SQLite/DuckDB files stand in for warehouses, stub transports for HTTP APIs, stub clients for LLMs, and temporary git repositories for diffs.

Area	Files	Tests	What they prove
Graph engine	`test_graph.py`	6	merge unifies code and data, save/load round-trip, tree shape, paths, depth limits, JSON-serialisable analysis
Python extraction	`test_python_extractor.py`	5	files/functions, **reversed** import and call impact, containment, and that an unrelated function is *not* affected
Provenance & exports	`test_provenance_exports.py`	6	call edges are `inferred` and excludable, dbt edges are `extracted`, GraphML/DOT/Cypher, hotspot ranking, schema drift, self-contained HTML
dbt	`test_dbt_extractor.py`, `test_dbt_compiled_lineage.py`, `test_jaffle_shop.py`	18	models/sources/exposures, downstream impact, upstream *not* affected, owners, column lineage from compiled SQL, catalog auto-detection, `select *` chains, and the same on dbt's real public project
SQL	`test_sql_extractor.py`, `test_sql_column_lineage.py`	6	table lineage, output columns, column edges through CTEs and aliases, rename propagation by lineage rather than name
Warehouse	`test_warehouse.py`, `test_warehouse_sqlite.py`	9	tables/columns/PKs/FKs, view lineage, `connect()` DSN forms, relationships summary, CLI, **system-schema exclusion**, DuckDB cleanliness
Columns	`test_column_impact.py`	4	column change reaches models and exposures, same-name heuristic is flagged, tree roots at the column
Git	`test_git_extractor.py`	2	uncommitted change maps to the exact function (not its neighbour); diff impact reaches the caller
Bridges	`test_bridge_detection.py`	4	SQL-in-code detection incl. placeholders, code→table edges, alias linking, CLI default
Orchestration	`test_airflow.py`, `test_lambda.py`, `test_js.py`	10	DAGs/tasks/dependencies/callables/SQL, SAM + serverless, handler→function, API events, JS functions/imports/SQL
Imports	`test_openlineage.py`, `test_lineage_file.py`, `test_datahub.py`	7	JSON array and NDJSON events, column-lineage facets, YAML/JSON lineage files, GraphQL import against a stub, error propagation
Analysis	`test_modeling.py`, `test_profiling.py`, `test_analyze.py`	9	star classification from FKs, inference without FKs, wide-table proposal, CLI/wiki/MCP, profiling stats + masking, the end-to-end `analyze` flow
Knowledge	`test_knowledge.py`	7	context pack, SQL/tests captured, wiki files, CLI, MCP tool, **ambiguity message**
AI	`test_ai_layer.py`, `test_llm_lineage.py`, `test_providers.py`	17	deterministic payload, refusal handling, suggestions applied with `llm` provenance, unknown tables refused, all three providers via stubs, Bedrock token clamping
Security	`test_security.py`	8	DSN redaction, sanitisation, sensitive-column detection, profile masking, DSN absent from graph/cache, HTML escaping of a malicious name, SQL literal escaping, prompt notices

Area	Files	Tests	What they prove
CLI & ops	`test_cli.py`, `test_cli_v2.py`, `test_mcp_maintenance.py`, `test_report.py`, `test_plugins.py`, `test_bom_files.py`, `test_lineage.py`	30	every command, `--update`, hooks, MCP tools, terminal rendering in both encodings, plugin registration, UTF-8 BOM handling, lineage rendering
impactgraph	`test_check.py`	6	diff→function→caller, all output formats, `--fail-on` exit code, passthrough, console encoding, UTF-8 files

`conftest.py` provides two fixtures — `py_project` (a tiny Python package) and `dbt_manifest` / `dbt_graph` (source → customer → dim_customer → fact_booking → two exposures) — which most tests build on.

38. CI/CD and packaging

`ci.yml` — on push to `main` and every PR: a 6-cell matrix (Ubuntu + Windows × Python 3.9, 3.11, 3.13) running `pip install -e .[dev]` and `pytest -q`, plus a build job producing wheel and sdist. impactgraph's CI installs datagraph from git first, so the two repos stay compatible before a release.

`publish.yml` — on a `v*` tag: test → `python -m build` → upload artefact → **GitHub Release** (softprops/action-gh-release with generated notes) → **PyPI via trusted publishing** (`pypa/gh-action-pypi-publish` with `id-token: write`, environment `pypi`, gated on the repository variable `PYPI_TRUSTED_PUBLISHER == 'true'`). **No API token exists anywhere** — PyPI verifies GitHub's OIDC identity (owner/repo/workflow/environment must match the pending publisher registered on PyPI).

Release procedure. Bump `version` in `pyproject.toml` and `__version__` in `__init__.py`, commit, `git tag vX.Y.Z && git push origin vX.Y.Z`. Nothing publishes without a tag.

`pyproject.toml`. setuptools, src-layout, `datagraph = "datagraph.cli:main"` console script; extras `sql` (sqlglot), `ai` (anthropic), `bedrock` (boto3), `mcp` (mcp, Python ≥3.10), `yaml` (PyYAML), `all`, `dev`. The distribution is named **`datagraph-core`** because PyPI rejects the bare name `datagraph`; the import name, CLI and repository are unchanged. impactgraph depends on `datagraph-core >= 0.8.4` and mirrors the extras.

39. Bugs found and fixed during development

Each is a test today; together they are the best summary of what the code has to survive.

Symptom	Root cause	Fix
A repo scanned to zero functions	Files saved with a UTF-8 **BOM**: `ast.parse` failed silently	Read everything with `utf-8-sig` (`test_bom_files.py`)
Crash on the first Windows run	cp1252 console cannot encode ⬢/📊	ASCII icon fallback in `report.py` (`test_report.py`)
Node named `[dbt] x` broke the report	rich interpreted `[dbt]` as markup	`rich.markup.escape` on every name
Column change reported nothing	Only table-level edges existed	Column propagation + real sqlglot column lineage (`test_column_impact.py`)
`select *` chains produced no column edges	No schema to expand the star	`qualify_with_schema` using catalog/manifest/warehouse (`test_jaffle_shop.py`)
`--update` always rebuilt	The graph and its cache were inside the fingerprinted inputs	`_outputs()` exclusion
`resolve()` returned the wrong node	A dbt model and a file shared a path	Fixed priority order (§6.4)
MySQL/DuckDB ingested system objects	Only two system schemas were excluded	`SYSTEM_CATALOGS` / `SYSTEM_SCHEMAS` (§13.3) — 0.8.4
"No node matches 'dim_customer'" for a name matching 11 nodes	Ambiguity indistinguishable from absence	Candidate list, tables before columns (§26.1) — 0.8.4
`impactgraph pr` crashed on Windows	Emoji in Markdown printed to cp1252	`safe_console_text` (§34.5) — 0.7.3
Bedrock rejected the request	Nova caps output at 10 000 tokens	Clamp + retry on the reported limit (§29)
Example tour aborted without PyYAML	Optional extractor raised	Per-extractor `try/except ImportError`, prints "skipped"

40. Known limits (stated, not hidden)

- Code languages: Python (via `ast`) and JS/TS (regex). No Java/Scala/Go yet.
- Call edges are name-resolved and therefore `inferred`; dynamic dispatch is undecidable statically.
- Column lineage needs parseable SQL or a catalog; otherwise the same-name heuristic (`inferred`) or the opt-in `llm` fallback applies.
- Dimensional classification is heuristic — it always shows confidence and reasons, and `--no-inferred` restricts it to declared foreign keys.
- Profiling is metadata-level and sampled, not a data-quality suite (use Great Expectations/Soda for that; datagraph tells you *where* to put those checks).
- `hotspots()` is O(V·E) and is only run on demand.
- Live tests so far: SQLite, DuckDB and dbt's jaffle_shop. Postgres/MySQL/Snowflake use the same `information_schema` path but have not been exercised against a live server in CI.

41. Extending the code

- **New extractor:** a class with `.extract() -> ImpactGraph` following the id conventions (§5). Ship it as a package with a `datagraph.extractors` entry point and it becomes a CLI flag (§20).
- **New node/edge type:** add to `NodeType` / `EdgeType`, add a direction in `IMPACT_DIRECTION`, a weight in `NODE_WEIGHTS`, a rule in `recommend_tests`, an icon in `report.py` / `html_report.py`.
- **New analysis:** a function taking `ImpactGraph` and returning plain data; wire a `_cmd_*` in `cli.py` and, if useful to assistants, a tool in `mcp_server.py`.
- **New LLM backend:** subclass `LLMProvider`, implement `complete`, register a name in `get_provider`.

Appendices

Appendix A — CLI reference (datagraph)

```
analyze  --warehouse DSN [--schemas a,b] [--database D] [--dialect X] [-o DIR]
         [--no-profile] [--sample N] [--no-top-values] [--no-inferred] [--json]
build    [--repo DIR] [--dbt-manifest F] [--dbt-catalog F] [--sql DIR] [--sql-dialect X]
         [--warehouse DSN] [--warehouse-schemas a,b] [--warehouse-database D]
         [--airflow DIR] [--lambda F] [--js DIR] [--openlineage F] [--lineage-file F]
         [--datahub URL] [--datahub-token T] [--datahub-query Q]
         [--no-alias-linking] [--no-column-lineage] [--update]
         [--llm-fallback] [--llm-provider P] [--llm-model M] [--llm-min-confidence C] [-o FILE]
impact   NODE... [--graph F] [--max-depth N] [--json] [--no-inferred] [--html F]
diff     [--repo DIR] [--base REF] [--head REF] + the impact options
lineage  NODE [--graph F] [--upstream-depth N] [--downstream-depth N] [--json] [--html F]
relationships [--search X] [--graph F] [--json] [--no-columns]
profile  --warehouse DSN [--graph F] [--tables a,b] [--sample N] [--no-top-values] [--json]
model    [--graph F] [--from-table T] [--no-inferred] [--json] [--mermaid F] [--markdown F]
context  NODE [--graph F] [--depth N]
wiki     [--graph F] [-o DIR] [--title T] [--with-files]
nodes    [--graph F] [--search X] [--type T]
paths    CHANGED TARGET [--graph F] [--json]
hotspots [--graph F] [--top N] [--no-inferred] [--json]
graph-diff OLD NEW [--json]
export   --format graphml|dot|cypher|json -o FILE [--graph F]
html     [NODE...] [--all] [--with-columns] -o FILE [--graph F]
watch    <build options> [--interval S]
hook-install --git-repo DIR --hook-cmd "..." [--hook post-commit]
explain  NODE... [--graph F] [--provider P] [--model M] [--max-depth N]
enrich   [--graph F] [--unparsed F] [--provider P] [--model M] [--min-confidence C] [--dry-run] [--json]
mcp      [--graph F]
plugins
```

Appendix B — CLI reference (impactgraph)

```
check [--repo DIR] [--code DIR] [--base REF] [--head REF] [--graph F] [--save-graph F] [--update]
      [all datagraph build inputs] [--max-depth N] [--no-inferred]
      [--format text|json|markdown] [--html F] [-o FILE] [--fail-on LEVEL] [--title T]
pr    same as check --format markdown
<anything else>  forwarded to datagraph
```

Exit codes: 0 = fine, 1 = risk at or above `--fail-on`, 2 = error.

Appendix C — Python API

```
# graph
ImpactGraph() / DataGraph()  .add_node .add_edge .merge .get_node .nodes .edges .edges_of
                             .find .resolve .impact .upstream .lineage .impact_tree .upstream_tree
                             .impact_paths .hotspots .link_table_aliases .subgraph
                             .to_graphml .to_dot .to_cypher .to_dict .save .load
diff_graphs(old, new)
Node(id, type, name, path, meta) · Edge(src, dst, type, meta) · NodeType · EdgeType
EXTRACTED · INFERRED · LLM

# extractors
PythonExtractor(root) · SqlExtractor(root, dialect) · DbtExtractor(manifest, column_lineage,
    dialect, catalog_path) · WarehouseExtractor(connection, database, schemas, dialect,
    view_lineage, foreign_keys, info_schema) · AirflowExtractor(root) · LambdaExtractor(template,
    code_root) · JsExtractor(root) · OpenLineageExtractor(path) · LineageFileExtractor(path) ·
    DataHubExtractor(server, token, query, ...) · connect(dsn)
collect_changes(repo, base, head) · changed_node_ids(graph, changes)
ExtractorPlugin(...) · register(plugin)

# analysis
analyze_impact(graph, changed, max_depth, include_inferred) -> ImpactAnalysis
relationships(graph, search, include_columns)
classify_tables(graph) · star_schema(graph) · propose_from_table(graph, table)
to_markdown(model) · to_mermaid(model) · fk_links(graph)
profile_warehouse(connection, graph, tables, sample, top_values, max_columns, log)
profile_summary(node)

# knowledge / output / services
context(graph, node_id, depth) · build_wiki(graph, out_dir, title, include_files)
render_analysis(graph, analysis) · render_lineage(graph, node_id, ...)
render_html(graph, analysis, title) · render_lineage_html(...) · render_graph_html(...)
build_tools(graph_path) · serve(graph_path)
fingerprint(paths) · is_up_to_date(graph_path, inputs) · write_cache(...) · watch(...) · install_hook(...)

# ai (optional)
explain_impact(analysis, model, api_key, provider) · suggest_lineage(graph, unparsed_sql, ...)
apply_suggestions(graph, suggestions, min_confidence) · schema_summary(graph)
get_provider(provider, model, api_key) · LLMProvider · AnthropicProvider · BedrockProvider ·
OpenAICompatibleProvider

# security
redact_dsn · sanitize_text · wrap_untrusted · is_sensitive_column · quote_ident · escape_literal ·
escape_script_json · UNTRUSTED_NOTICE

# impactgraph
check(repo, base, head, code, graph, graph_path, inputs, save_graph, update, max_depth,
      include_inferred, quiet) -> CheckResult
to_markdown(result, title) · safe_console_text(text, stream) · CheckResult
```

Appendix D — Environment variables

Variable	Used by	Meaning
`ANTHROPIC_API_KEY`	Anthropic provider	API key (read by the SDK)
`AWS_REGION` / `AWS_DEFAULT_REGION`, standard AWS chain	Bedrock provider	Region and credentials (profile, SSO, instance role)
`DATAGRAPH_LLM_PROVIDER`	`get_provider`	`anthropic` | `bedrock` | `openai`
`DATAGRAPH_LLM_MODEL`	`get_provider`	Default model id
`DATAGRAPH_LLM_API_KEY` / `OPENAI_API_KEY`	OpenAI-compatible provider	Bearer token
`DATAGRAPH_LLM_BASE_URL` / `OPENAI_BASE_URL`	OpenAI-compatible provider	Endpoint, e.g. `http://localhost:11434/v1`
`DATAGRAPH_LLM_MAX_TOKENS`	Bedrock provider	Output cap (default 8000)
`DATAHUB_TOKEN`	DataHub extractor	Bearer token

Appendix E — Complete file inventory

datagraph: `README.md`, `CONTRIBUTING.md`, `LICENSE`, `pyproject.toml`, `.gitignore`, `.github/workflows/{ci,publish}.yml`, `skills/datagraph/SKILL.md`, `examples/example_datagraph.py`, `examples/demo/{build_demo.py,manifest.json,app/booking_api.py}`, `examples/jaffle_shop/{manifest.json,catalog.json}`, `examples/mcp/claude-mcp.json`, `docs/{datagraph-documentation,datagraph-learning-guide}.{docx,pdf}`, `docs/TECHNICAL_REFERENCE.md` (this document), `docs/images/*.png`, `src/datagraph/**` (32 modules), `tests/**` (29 files).

impactgraph: `README.md`, `CONTRIBUTING.md`, `LICENSE`, `pyproject.toml`, `.gitignore`, `action.yml`, `.github/workflows/{ci,publish}.yml`, `skills/impactgraph/SKILL.md`, `examples/{example_impactgraph.py,github-workflow-impact.yml}`, `src/impactgraph/{__init__,core,cli}.py`, `tests/test_check.py`.

Appendix F — Glossary

Blast radius — the set of nodes reachable from a change, with the depth at which each is reached. **Bus matrix** — Kimball's facts × dimensions grid; shared columns are conformed dimensions. **Conformed dimension** — a dimension used by two or more facts. **Degenerate dimension** — a high-cardinality attribute kept on the fact rather than its own table. **Grain** — what one row of a fact represents. **Provenance** — how an edge was obtained: `extracted`, `inferred` or `llm`. **SCD** — slowly changing dimension; type 1 overwrites, type 2 keeps history. **Semi-additive measure** — a measure that cannot be summed across time (a balance, a stock level). **Trusted publishing** — PyPI uploads authenticated by GitHub's OIDC identity instead of an API token.

Generated for datagraph-core 0.8.4 and impactgraph 0.7.3. Line numbers refer to those releases. The authoritative source is always the code: github.com/sumit-gupta03/datagraph.